Java | Python

Warmup-1 > sleep_in

prev | next | chance

The parameter weekday is True if it is a weekday, and the parameter vacation is True if we are on vacation. We sleep in if it is not a weekday or we're on vacation. Return True if we sleep in.

sleep_in(False, False) → True
sleep_in(True, False) → False
sleep_in(False, True) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def sleep_in(weekday, vacation):
  if not weekday or vacation:
    return True
  else:
    return False
```

Expected	Run	
sleep_in(False, False) → True	True	OK
sleep_in(True, False) → False	False	OK
sleep_in(False, True) → True	True	OK
sleep_in(True, True) → True	True	OK

All Correct

next | chance

Python > Warmup-1

done page

Code is saved so long as this session is active. Create an account above to save code past this session.

Your progress graph for this problem

Java | Python

Warmup-1 > monkey_trouble

prev | next | chance

We have two monkeys, a and b, and the parameters a_smile and b_smile indicate if each is smiling. We are in trouble if they are both smiling or if neither of them is smiling. Return True if we are in trouble.

monkey_trouble(True, True) → True
monkey_trouble(False, False) → True
monkey_trouble(True, False) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def monkey_trouble(a_smile, b_smile):
  if a_smile and b_smile:
    return True
  if not a_smile and not b_smile:
    return True
  return False
```

Expected	Run	
monkey_trouble(True, True) → True	True	OK
monkey_trouble(False, False) → True	True	OK
monkey_trouble(True, False) → False	False	OK
monkey_trouble(False, True) → False	False	OK

All Correct

next | chance

Python > Warmup-1

done page

Code is saved so long as this session is active. Create an acco to save code past this session.

Your progress graph for this problem

Java | Python

Warmup-1 > sum_double

prev | next | chance

Given two int values, return their sum. Unless the two values are the same, then return double their sum.

sum_double(1, 2) → 3
sum_double(3, 2) → 5
sum_double(2, 2) → 8

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def sum_double(a, b):
  if a !=b :
    return a+b
  return 2*(a+b)
```

Expected	Run		
sum_double(1, 2) → 3	3	OK	
sum_double(3, 2) → 5	5	OK	
sum_double(2, 2) → 8	8	OK	
sum_double(-1, 0) → -1	-1	OK	
sum_double(3, 3) → 12	12	OK	
sum_double(0, 0) → 0	0	OK	
sum_double(0, 1) → 1	1	OK	
sum_double(3, 4) → 7	7	OK	

All Correct

next | chance

Python > Warmup-1

done page

Code is saved so long as this session is active. Create an account above

Java | Python

Warmup-1 > diff21

prev | next | chance

Given an int n, return the absolute difference between n and 21, except return double the absolute difference if n is over 21.

diff21(19) → 2
diff21(10) → 11
diff21(21) → 0

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def diff21(n):
  if n<=21:
    return 21-n
  return (n-21)*2
```

Expected	Run		
diff21(19) → 2	2	OK	
diff21(10) → 11	11	OK	
diff21(21) → 0	0	OK	
diff21(22) → 2	2	OK	
diff21(25) → 8	8	OK	
diff21(30) → 18	18	OK	
diff21(0) → 21	21	OK	
diff21(1) → 20	20	OK	
diff21(2) → 19	19	OK	
diff21(-1) → 22	22	OK	
diff21(-2) → 23	23	OK	
diff21(50) → 58	58	OK	

All Correct

Java | Python

Warmup-1 > parrot_trouble

prev | next | chance

We have a loud talking parrot. The "hour" parameter is the current hour time in the range 0..23. We are in trouble if the parrot is talking and the hour is before 7 or after 20. Return True if we are in trouble.

parrot_trouble(True, 6) → True
parrot_trouble(True, 7) → False
parrot_trouble(False, 6) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def parrot_trouble(talking, hour):
  return (talking and (hour < 7 or hour > 20))
```

Expected	Run	
parrot_trouble(True, 6) → True	True	OK
parrot_trouble(True, 7) → False	False	OK
parrot_trouble(False, 6) → False	False	OK
parrot_trouble(True, 21) → True	True	OK
parrot_trouble(False, 21) → False	False	OK
parrot_trouble(False, 20) → False	False	OK
parrot_trouble(True, 23) → True	True	OK
parrot_trouble(False, 23) → False	False	OK
parrot_trouble(True, 20) → False	False	OK
parrot_trouble(False, 12) → False	False	OK

All Correct

next | chance

Python > Warmup-1

done page

Code is saved so long as this session is active. Create an account abov

Java | Python

Warmup-1 > makes10

prev | next | chance

Given 2 ints, a and b, return True if one if them is 10 or if their sum is 10.

makes10(9, 10) → True
makes10(9, 9) → False
makes10(1, 9) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def makes10(a, b):
  if a == 10 or b == 10 or a+b ==10:
    return True
  return False
```

Expected	Run	
makes10(9, 10) → True	True	OK
makes10(9, 9) → False	False	OK
makes10(1, 9) → True	True	OK
makes10(10, 1) → True	True	OK
makes10(10, 10) → True	True	OK
makes10(8, 2) → True	True	OK
makes10(8, 3) → False	False	OK
makes10(10, 42) → True	True	OK
makes10(12, -2) → True	True	OK

All Correct

next | chance

Python > Warmup-1

Java | Python

Warmup-1 > near_hundred

prev | next | chance

Given an int n, return True if it is within 10 of 100 or 200. Note: abs(num) computes the absolute value of a number.

near_hundred(93) → True
near_hundred(90) → True
near_hundred(89) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def near_hundred(n):
  return ((abs(100 - n) <= 10) or (abs(200 - n) <= 10))
```

Expected	Run	
near_hundred(93) → True	True	OK
near_hundred(90) → True	True	OK
near_hundred(89) → False	False	OK
near_hundred(110) → True	True	OK
near_hundred(111) → False	False	OK
near_hundred(121) → False	False	OK
near_hundred(-101) → False	False	OK
near_hundred(-209) → False	False	OK
near_hundred(190) → True	True	OK
near_hundred(209) → True	True	OK
near_hundred(0) → False	False	OK
near_hundred(5) → False	False	OK
near_hundred(-50) → False	False	OK
near_hundred(191) → True	True	OK
near_hundred(189) → False	False	OK
near_hundred(200) → True	True	OK
near_hundred(210) → True	True	OK

Go

Warmup-1 > pos_neg

rev | next | chance

Given 2 int values, return True if one is negative and one is positive. Except if the parameter "negative" is True, then return True only if both are negative.

pos_neg(1, -1, False) → True
pos_neg(-1, 1, False) → True
pos_neg(-4, -5, True) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def pos_neg(a, b, negative):
  if negative:
    return (a < 0 and b < 0)
  else:
    return ((a < 0 and b > 0) or (a > 0 and b < 0))
```

Expected	Run	
pos_neg(1, -1, False) → True	True	OK
pos_neg(-1, 1, False) → True	True	OK
pos_neg(-4, -5, True) → True	True	OK
pos_neg(-4, -5, False) → False	False	OK
pos_neg(-4, 5, False) → True	True	OK
pos_neg(-4, 5, True) → False	False	OK
pos_neg(1, 1, False) → False	False	OK
pos_neg(-1, -1, False) → False	False	OK
pos_neg(1, -1, True) → False	False	OK
pos_neg(-1, 1, True) → False	False	OK
pos_neg(1, 1, True) → False	False	OK
pos_neg(-1, -1, True) → True	True	OK

Warmup-1 > not_string

prev | next | chance

Given a string, return a new string where "not " has been added to the front. However, if the string already begins with "not", return the string unchanged.

not_string('candy') → 'not candy'
not_string('x') → 'not x'
not_string('not bad') → 'not bad'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def not_string(str):
  if len(str) >= 3 and str[:3] == "not":
    return str
  return "not " + str
```

Expected	Run	
not_string('candy') → 'not candy'	'not candy'	OK
not_string('x') → 'not x'	'not x'	OK
not_string('not bad') → 'not bad'	'not bad'	OK
not_string('bad') → 'not bad'	'not bad'	OK
not_string('not') → 'not'	'not'	OK
not_string('is not') → 'not is not'	'not is not'	OK
not_string('no') → 'not no'	'not no'	OK

All Correct

next | chance

Java | Python

Warmup-1 > missing_char

prev | next | chance

Given a non-empty string and an int n, return a new string where the char at index n has been removed. The value of n will be a valid index of a char in the original string (i.e. n will be in the range 0..len(str)-1 inclusive).

missing_char('kitten', 1) → 'ktten'
missing_char('kitten', 0) → 'itten'
missing_char('kitten', 4) → 'kittn'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def missing_char(str, n):
  front = str[:n]
  back = str[n+1:]
  return front + back
```

Expected	Run		
missing_char('kitten', 1) → 'ktten'	'ktten'	OK	
missing_char('kitten', 0) → 'itten'	'itten'	OK	
missing_char('kitten', 4) → 'kittn'	'kittn'	OK	
missing_char('Hi', 0) → 'i'	'i'	OK	
missing_char('Hi', 1) → 'H'	'H'	OK	
missing_char('code', 0) → 'ode'	'ode'	OK	
missing_char('code', 1) → 'cde'	'cde'	OK	
missing_char('code', 2) → 'coe'	'coe'	OK	
missing_char('code', 3) → 'cod'	'cod'	OK	
missing_char('chocolate', 8) → 'chocolat'	'chocolat'	OK	

All Correct

Warmup-1 > front_back

prev | next | chance

Given a string, return a new string where the first and last chars have been exchanged.

front_back('code') → 'eodc'
front_back('a') → 'a'
front_back('ab') → 'ba'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front_back(str):
  if len(str) <= 1:
    return str

  mid = str[1:len(str)-1]
  return str[len(str)-1] + mid + str[0]
```

Expected	Run		
front_back('code') → 'eodc'	'eodc'	OK	
front_back('a') → 'a'	'a'	OK	
front_back('ab') → 'ba'	'ba'	OK	
front_back('abc') → 'cba'	'cba'	OK	
front_back('') → ''	''	OK	
front_back('Chocolate') → 'ehocolatC'	'ehocolatC'	OK	
front_back('aavJ') → 'Java'	'Java'	OK	
front_back('hello') → 'oellh'	'oellh'	OK	

All Correct

next | chance

Python > Warmup-1

done page

Code is saved so long as this session is active. Create an account above to save code past this session.

Java | Python

Warmup-1 > front3

prev | next | chance

Given a string, we'll say that the front is the first 3 chars of the string. If the string length is less than 3, the front is whatever is there. Return a new string which is 3 copies of the front.

front3('Java') → 'JavJavJav'
front3('Chocolate') → 'ChoChoCho'
front3('abc') → 'abcabcabc'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front3(str):
  front_end = 3
  if len(str) < front_end:
    front_end = len(str)
  front = str[:front_end]
  return front + front + front
```

Expected	Run	
front3('Java') → 'JavJavJav'	'JavJavJav'	OK
front3('Chocolate') → 'ChoChoCho'	'ChoChoCho'	OK
front3('abc') → 'abcabcabc'	'abcabcabc'	OK
front3('abcXYZ') → 'abcabcabc'	'abcabcabc'	OK
front3('ab') → 'ababab'	'ababab'	OK
front3('a') → 'aaa'	'aaa'	OK
front3('') → ''	''	OK

All Correct

next | chance

Warmup-2 > string_times

prev | next | chance

Given a string and a non-negative int n, return a larger string that is n copies of the original string.

string_times('Hi', 2) → 'HiHi'
string_times('Hi', 3) → 'HiHiHi'
string_times('Hi', 1) → 'Hi'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_times(str, n):
  result = ""
  for i in range(n):
    result = result + str
  return result
```

Expected	Run	
string_times('Hi', 2) → 'HiHi'	'HiHi'	OK
string_times('Hi', 3) → 'HiHiHi'	'HiHiHi'	OK
string_times('Hi', 1) → 'Hi'	'Hi'	OK
string_times('Hi', 0) → ''	''	OK
string_times('Hi', 5) → 'HiHiHiHiHi'	'HiHiHiHiHi'	OK
string_times('Oh Boy!', 2) → 'Oh Boy!Oh Boy!'	'Oh Boy!Oh Boy!'	OK
string_times('x', 4) → 'xxxx'	'xxxx'	OK
string_times('', 4) → ''	''	OK
string_times('code', 2) → 'codecode'	'codecode'	OK
string_times('code', 3) → 'codecodecode'	'codecodecode'	OK

All Correct

next | chance

Python > Warmup-2

Java | Python

Warmup-2 > front_times

prev | next | chance

Given a string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front;

front_times('Chocolate', 2) → 'ChoCho'
front_times('Chocolate', 3) → 'ChoChoCho'
front_times('Abc', 3) → 'AbcAbcAbc'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def front_times(str, n):
  front_len = 3
  if front_len > len(str):
    front_len = len(str)
  front = str[:front_len]

  result = ""
  for i in range(n):
    result = result + front
  return result
```

Expected	Run	
front_times('Chocolate', 2) → 'ChoCho'	'ChoCho'	OK
front_times('Chocolate', 3) → 'ChoChoCho'	'ChoChoCho'	OK
front_times('Abc', 3) → 'AbcAbcAbc'	'AbcAbcAbc'	OK
front_times('Ab', 4) → 'AbAbAbAb'	'AbAbAbAb'	OK
front_times('A', 4) → 'AAAA'	'AAAA'	OK
front_times('', 4) → ''	''	OK
front_times('Abc', 0) → ''	''	OK

All Correct

next | chance

Python > Warmup-2

done page

Warmup-2 > string_bits

prev | next | chance

Given a string, return a new string made of every other char starting with the first, so "Hello" yields "Hlo".

string_bits('Hello') → 'Hlo'
string_bits('Hi') → 'H'
string_bits('Heeololeo') → 'Hello'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_bits(str):
  result = ""
  for i in range(len(str)):
    if i % 2 == 0:
      result = result + str[i]
  return result
```

Expected	Run	
string_bits('Hello') → 'Hlo'	'Hlo'	OK
string_bits('Hi') → 'H'	'H'	OK
string_bits('Heeololeo') → 'Hello'	'Hello'	OK
string_bits('HiHiHi') → 'HHH'	'HHH'	OK
string_bits('') → ''	''	OK
string_bits('Greetings') → 'Getns'	'Getns'	OK
string_bits('Chocoate') → 'Coot'	'Coot'	OK
string_bits('pi') → 'p'	'p'	OK
string_bits('Hello Kitten') → 'HloKte'	'HloKte'	OK
string_bits('hxaxpxpxy') → 'happy'	'happy'	OK

All Correct

next | chance

Python > Warmup-2

done page

Warmup-2 > string_splosion

prev | next | chance

Given a non-empty string like "Code" return a string like "CCoCodCode".

string_splosion('Code') → 'CCoCodCode'
string_splosion('abc') → 'aababc'
string_splosion('ab') → 'aab'

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_splosion(str):
  result = ""
  for i in range(len(str)):
    result = result + str[:i+1]
  return result
```

Expected	Run	
string_splosion('Code') → 'CCoCodCode'	'CCoCodCode'	OK
string_splosion('abc') → 'aababc'	'aababc'	OK
string_splosion('ab') → 'aab'	'aab'	OK
string_splosion('x') → 'x'	'x'	OK
string_splosion('fade') → 'ffafadfade'	'ffafadfade'	OK
string_splosion('There') → 'TThTheTherThere'	'TThTheTherThere'	OK
string_splosion('Kitten') → 'KKiKitKittKitteKitten'	'KKiKitKittKitteKitten'	OK
string_splosion('Bye') → 'BByBye'	'BByBye'	OK
string_splosion('Good') → 'GGoGooGood'	'GGoGooGood'	OK
string_splosion('Bad') → 'BBaBad'	'BBaBad'	OK

All Correct

next | chance

Warmup-2 > last2

prev | next | chance

Given a string, return the count of the number of times that a substring length 2 appears in the string and also as the last 2 chars of the string, so "hixxxhi" yields 1 (we won't count the end substring).

last2('hixxhi') → 1
last2('xaxxaxaxx') → 1
last2('axxxaaxx') → 2

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def last2(str):
  if len(str) < 2:
    return 0
  last2 = str[len(str)-2:]
  count = 0
  for i in range(len(str)-2):
    sub = str[i:i+2]
    if sub == last2:
      count = count + 1

  return count
```

Expected	Run	
last2('hixxhi') → 1	1	OK
last2('xaxxaxaxx') → 1	1	OK
last2('axxxaaxx') → 2	2	OK
last2('xxaxxaxxaxx') → 3	3	OK
last2('xaxaxaxx') → 0	0	OK
last2('xxxx') → 2	2	OK
last2('13121312') → 1	1	OK
last2('11212') → 1	1	OK
last2('13121311') → 0	0	OK
last2('1717171') → 2	2	OK
last2('hi') → 0	0	OK
last2('h') → 0	0	OK
last2('') → 0	0	OK

Warmup-2 > array_count9

prev | next | chance

Given an array of ints, return the number of 9's in the array.

array_count9([1, 2, 9]) → 1
array_count9([1, 9, 9]) → 2
array_count9([1, 9, 9, 3, 9]) → 3

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array_count9(nums):
  count =0;
  for i in range(len(nums)):
    if nums[i]==9:
      count+=1
  return count
```

Expected	Run	
array_count9([1, 2, 9]) → 1	1	OK
array_count9([1, 9, 9]) → 2	2	OK
array_count9([1, 9, 9, 3, 9]) → 3	3	OK
array_count9([1, 2, 3]) → 0	0	OK
array_count9([]) → 0	0	OK
array_count9([4, 2, 4, 3, 1]) → 0	0	OK
array_count9([9, 2, 4, 3, 1]) → 1	1	OK

All Correct

next | chance

Python > Warmup-2

done page

Code is saved so long as this session is active. Create an account above to save code past this session.

Your progress graph for this problem

Warmup-2 > array_front9

prev | next | chance

Given an array of ints, return True if one of the first 4 elements in the array is a 9. The array length may be less than 4.

array_front9([1, 2, 9, 3, 4]) → True
array_front9([1, 2, 3, 4, 9]) → False
array_front9([1, 2, 3, 4, 5]) → False

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array_front9(nums):
  end = len(nums)
  if end > 4:
   end = 4

  for i in range(end):
   if nums[i] == 9:
     return True
  return False
```

Expected	Run	
array_front9([1, 2, 9, 3, 4]) → True	True	OK
array_front9([1, 2, 3, 4, 9]) → False	False	OK
array_front9([1, 2, 3, 4, 5]) → False	False	OK
array_front9([9, 2, 3]) → True	True	OK
array_front9([1, 9, 9]) → True	True	OK
array_front9([1, 2, 3]) → False	False	OK
array_front9([1, 9]) → True	True	OK
array_front9([5, 5]) → False	False	OK
array_front9([2]) → False	False	OK
array_front9([9]) → True	True	OK
array_front9([]) → False	False	OK
array_front9([3, 9, 2, 3, 3]) → True	True	OK

All Correct

next | chance

Python > Warmup-2

CodingBat code practice

Java | Python

Warmup-2 > array123

prev | next | chance

Given an array of ints, return True if the sequence of numbers 1, 2, 3 appears in the array somewhere.

array123([1, 1, 2, 3, 1]) → True
array123([1, 1, 2, 4, 1]) → False
array123([1, 1, 2, 1, 2, 3]) → True

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def array123(nums):
  for i in range(len(nums)-2):
    if nums[i]==1 and nums[i+1]==2 and nums[i+2]==3:
      return True
  return False
```

Go

Expected	Run		
array123([1, 1, 2, 3, 1]) → True	True	OK	
array123([1, 1, 2, 4, 1]) → False	False	OK	
array123([1, 1, 2, 1, 2, 3]) → True	True	OK	
array123([1, 1, 2, 1, 2, 1]) → False	False	OK	
array123([1, 2, 3, 1, 2, 3]) → True	True	OK	
array123([1, 2, 3]) → True	True	OK	
array123([1, 1, 1]) → False	False	OK	
array123([1, 2]) → False	False	OK	
array123([1]) → False	False	OK	
array123([]) → False	False	OK	

All Correct

next | chance

Python > Warmup-2

done page

Code is saved so long as this session is active. Create an account above

Java | Python

Warmup-2 > string_match

prev | next | chance

Given 2 strings, a and b, return the number of the positions where they contain the same length 2 substring. So "xxcaazz" and "xxbaaz" yields 3, since the "xx", "aa", and "az" substrings appear in the same place in both strings.

string_match('xxcaazz', 'xxbaaz') → 3
string_match('abc', 'abc') → 2
string_match('abc', 'axc') → 0

Go ...Save, Compile, Run (ctrl-enter) Show Solution

```
def string_match(a, b):
  shorter = min(len(a), len(b))
  count = 0
  for i in range(shorter-1):
    a_sub = a[i:i+2]
    b_sub = b[i:i+2]
    if a_sub == b_sub:
      count = count + 1

  return count
```

Expected	Run		
string_match('xxcaazz', 'xxbaaz') → 3	3	OK	
string_match('abc', 'abc') → 2	2	OK	
string_match('abc', 'axc') → 0	0	OK	
string_match('hello', 'he') → 1	1	OK	
string_match('he', 'hello') → 1	1	OK	
string_match('h', 'hello') → 0	0	OK	
string_match('', 'hello') → 0	0	OK	
string_match('aabbccdd', 'abbbxxd') → 1	1	OK	
string_match('aaxxaaxx', 'iaxxai') → 3	3	OK	
string_match('iaxxai', 'aaxxaaxx') → 3	3	OK	

All Correct

next | chance

Python > Warmup-2

done page

String-1 > hello_name

prev | next | chance

Given a string name, e.g. "Bob", return a greeting of the form "Hello Bob!".

hello_name('Bob') → 'Hello Bob!'
hello_name('Alice') → 'Hello Alice!'
hello_name('X') → 'Hello X!'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def hello_name(name):
    return "Hello " + name + "!"
```

Expected	Run		
hello_name('Bob') → 'Hello Bob!'	'Hello Bob!'	OK	
hello_name('Alice') → 'Hello Alice!'	'Hello Alice!'	OK	
hello_name('X') → 'Hello X!'	'Hello X!'	OK	
hello_name('Dolly') → 'Hello Dolly!'	'Hello Dolly!'	OK	
hello_name('Alpha') → 'Hello Alpha!'	'Hello Alpha!'	OK	
hello_name('Omega') → 'Hello Omega!'	'Hello Omega!'	OK	
hello_name('Goodbye') → 'Hello Goodbye!'	'Hello Goodbye!'	OK	
hello_name('ho ho ho') → 'Hello ho ho ho!'	'Hello ho ho ho!'	OK	
hello_name('xyz!') → 'Hello xyz!!'	'Hello xyz!!'	OK	
hello_name('Hello') → 'Hello Hello!'	'Hello Hello!'	OK	
other tests		OK	

String-1 > make_abba

prev | next | chance

Given two strings, a and b, return the result of putting them together in the order abba, e.g. "Hi" and "Bye" returns "HiByeByeHi".

make_abba('Hi', 'Bye') → 'HiByeByeHi'
make_abba('Yo', 'Alice') → 'YoAliceAliceYo'
make_abba('What', 'Up') → 'WhatUpUpWhat'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def make_abba(a, b):
  return a+b+b+a
```

Expected	Run		
make_abba('Hi', 'Bye') → 'HiByeByeHi'	'HiByeByeHi'	OK	
make_abba('Yo', 'Alice') → 'YoAliceAliceYo'	'YoAliceAliceYo'	OK	
make_abba('What', 'Up') → 'WhatUpUpWhat'	'WhatUpUpWhat'	OK	
make_abba('aaa', 'bbb') → 'aaabbbbbbaaa'	'aaabbbbbbaaa'	OK	
make_abba('x', 'y') → 'xyyx'	'xyyx'	OK	
make_abba('x', '') → 'xx'	'xx'	OK	
make_abba('', 'y') → 'yy'	'yy'	OK	
make_abba('Bo', 'Ya') → 'BoYaYaBo'	'BoYaYaBo'	OK	
make_abba('Ya', 'Ya') → 'YaYaYaYa'	'YaYaYaYa'	OK	
other tests		OK	

All Correct

Good job -- problem solved. You can see our solution as an alternative.

String-1 > make_tags

prev | next | chance

The web is built with HTML strings like "Yay" which draws Yay as italic text. In this example, the "i" tag makes and which surround the word "Yay". Given tag and word strings, create the HTML string with tags around the word, e.g. "Yay".

make_tags('i', 'Yay') → 'Yay'
make_tags('i', 'Hello') → 'Hello'
make_tags('cite', 'Yay') → '<cite>Yay</cite>'

Go ...Save, Compile, Run (ctrl-enter)

```
def make_tags(tag, word):
    return "<" + tag + ">" + word + "</" + tag + ">"
```

Expected	Run		
make_tags('i', 'Yay') → 'Yay'	'Yay'	OK	
make_tags('i', 'Hello') → 'Hello'	'Hello'	OK	
make_tags('cite', 'Yay') → '<cite>Yay</cite>'	'<cite>Yay</cite>'	OK	
make_tags('address', 'here') → '<address>here</address>'	'<address>here</address>'	OK	
make_tags('body', 'Heart') → '<body>Heart</body>'	'<body>Heart</body>'	OK	
make_tags('i', 'i') → 'i'	'i'	OK	
make_tags('i', '') → ''	''	OK	
other tests		OK	

All Correct

next | chance

Python > String-1

Java | Python

String-1 > make_out_word

prev | next | chance

Given an "out" string length 4, such as "<<>>", and a word, return a new string where the word is in the middle of the out string, e.g. "<<word>>".

make_out_word('<<>>', 'Yay') → '<<Yay>>'
make_out_word('<<>>', 'WooHoo') → '<<WooHoo>>'
make_out_word('[[]]', 'word') → '[[word]]'

Go ...Save, Compile, Run (ctrl-enter)

```
def make_out_word(out, word):
    return out[:2] + word + out[2:]
```

Expected	Run	
make_out_word('<<>>', 'Yay') → '<<Yay>>'	'<<Yay>>'	OK
make_out_word('<<>>', 'WooHoo') → '<<WooHoo>>'	'<<WooHoo>>'	OK
make_out_word('[[]]', 'word') → '[[word]]'	'[[word]]'	OK
make_out_word('HHoo', 'Hello') → 'HHHellooo'	'HHHellooo'	OK
make_out_word('abyz', 'YAY') → 'abYAYyz'	'abYAYyz'	OK
other tests		OK

All Correct

next | chance

Python > String-1

done page

String-1 > extra_end

prev | next | chance

Given a string, return a new string made of 3 copies of the last 2 chars of the original string. The string length will be at least 2.

extra_end('Hello') → 'lololo'
extra_end('ab') → 'ababab'
extra_end('Hi') → 'HiHiHi'

Go ...Save, Compile, Run (ctrl-enter)

```
def extra_end(str):
    return str[-2:] * 3
```

Expected	Run	
extra_end('Hello') → 'lololo'	'lololo'	OK
extra_end('ab') → 'ababab'	'ababab'	OK
extra_end('Hi') → 'HiHiHi'	'HiHiHi'	OK
extra_end('Candy') → 'dydydy'	'dydydy'	OK
extra_end('Code') → 'dedede'	'dedede'	OK
other tests		OK

All Correct

String-1 > first_two

prev | next | chance

Given a string, return the string made of its first two chars, so the String "Hello" yields "He". If the string is shorter than length 2, return whatever there is, so "X" yields "X", and the empty string "" yields the empty string "".

first_two('Hello') → 'He'
first_two('abcdefg') → 'ab'
first_two('ab') → 'ab'

Go ...Save, Compile, Run (ctrl-enter)

```
def first_two(s):
    return s[:2]
```

Expected	Run	
first_two('Hello') → 'He'	'He'	OK
first_two('abcdefg') → 'ab'	'ab'	OK
first_two('ab') → 'ab'	'ab'	OK
first_two('a') → 'a'	'a'	OK
first_two('') → ''	''	OK
first_two('Kitten') → 'Ki'	'Ki'	OK
first_two('hi') → 'hi'	'hi'	OK
first_two('hiya') → 'hi'	'hi'	OK
other tests		OK

All Correct

Good job -- problem solved. You can see our solution as an alt

See Our Solution

next | chance

String-1 > first_half

prev | next | chance

Given a string of even length, return the first half. So the string "WooHoo" yields "Woo".

first_half('WooHoo') → 'Woo'
first_half('HelloThere') → 'Hello'
first_half('abcdef') → 'abc'

Go ...Save, Compile, Run (ctrl-enter)

```
def first_half(s):
    return s[:len(s)//2]
```

Expected	Run	
first_half('WooHoo') → 'Woo'	'Woo'	OK
first_half('HelloThere') → 'Hello'	'Hello'	OK
first_half('abcdef') → 'abc'	'abc'	OK
first_half('ab') → 'a'	'a'	OK
first_half('') → ''	''	OK
first_half('0123456789') → '01234'	'01234'	OK
first_half('kitten') → 'kit'	'kit'	OK
other tests		OK

All Correct

String-1 > without_end

prev | next | chance

Given a string, return a version without the first and last char, so "Hello" yields "ell". The string length will be at least 2.

without_end('Hello') → 'ell'
without_end('java') → 'av'
without_end('coding') → 'odin'

Go ...Save, Compile, Run (ctrl-enter)

```
def without_end(s):
    return s[1:-1]
```

Expected	Run	
without_end('Hello') → 'ell'	'ell'	OK
without_end('java') → 'av'	'av'	OK
without_end('coding') → 'odin'	'odin'	OK
without_end('code') → 'od'	'od'	OK
without_end('ab') → ''	''	OK
without_end('Chocolate!') → 'hocolate'	'hocolate'	OK
without_end('kitten') → 'itte'	'itte'	OK
without_end('woohoo') → 'ooho'	'ooho'	OK
other tests		OK

All Correct

next | chance

Python > String-1

done page

Code is saved so long as this session is active. Create an account above

String-1 > combo_string

prev | next | chance

Given 2 strings, a and b, return a string of the form short+long+short, with the shorter string on the outside and the longer string on the inside. The strings will not be the same length, but they may be empty (length 0).

combo_string('Hello', 'hi') → 'hiHellohi'
combo_string('hi', 'Hello') → 'hiHellohi'
combo_string('aaa', 'b') → 'baaab'

Go ...Save, Compile, Run (ctrl-enter)

```
def combo_string(a, b):
    if len(a) < len(b):
        return a + b + a
    else:
        return b + a + b
```

Expected	Run	
combo_string('Hello', 'hi') → 'hiHellohi'	'hiHellohi'	OK
combo_string('hi', 'Hello') → 'hiHellohi'	'hiHellohi'	OK
combo_string('aaa', 'b') → 'baaab'	'baaab'	OK
combo_string('b', 'aaa') → 'baaab'	'baaab'	OK
combo_string('aaa', '') → 'aaa'	'aaa'	OK
combo_string('', 'bb') → 'bb'	'bb'	OK
combo_string('aaa', '1234') → 'aaa1234aaa'	'aaa1234aaa'	OK
combo_string('aaa', 'bb') → 'bbaaabb'	'bbaaabb'	OK
combo_string('a', 'bb') → 'abba'	'abba'	OK
combo_string('bb', 'a') → 'abba'	'abba'	OK
combo_string('xyz', 'ab') → 'abxyzab'	'abxyzab'	OK
other tests		OK

All Correct

next | chance

Python > String-1

done page

Java | Python

String-1 > non_start

prev | next | chance

Given 2 strings, return their concatenation, except omit the first char of each. The strings will be at least length 1.

non_start('Hello', 'There') → 'ellohere'
non_start('java', 'code') → 'avaode'
non_start('shotl', 'java') → 'hotlava'

Go ...Save, Compile, Run (ctrl-enter)

```
def non_start(a, b):
    return a[1:] + b[1:]
```

Expected	Run		
non_start('Hello', 'There') → 'ellohere'	'ellohere'	OK	
non_start('java', 'code') → 'avaode'	'avaode'	OK	
non_start('shotl', 'java') → 'hotlava'	'hotlava'	OK	
non_start('ab', 'xy') → 'by'	'by'	OK	
non_start('ab', 'x') → 'b'	'b'	OK	
non_start('x', 'ac') → 'c'	'c'	OK	
non_start('a', 'x') → ''	''	OK	
non_start('kit', 'kat') → 'itat'	'itat'	OK	
non_start('mart', 'dart') → 'artart'	'artart'	OK	
other tests		OK	

All Correct

next | chance

Python > String-1

Java | Python

String-1 > left2

prev | next | chance

Given a string, return a "rotated left 2" version where the first 2 chars are moved to the end. The string length will be at least 2.

left2('Hello') → 'lloHe'
left2('java') → 'vaja'
left2('Hi') → 'Hi'

Go ...Save, Compile, Run (ctrl-enter)

```
def left2(s):
    return s[2:] + s[:2]
```

Go

Expected	Run		
left2('Hello') → 'lloHe'	'lloHe'	OK	
left2('java') → 'vaja'	'vaja'	OK	
left2('Hi') → 'Hi'	'Hi'	OK	
left2('code') → 'deco'	'deco'	OK	
left2('cat') → 'tca'	'tca'	OK	
left2('12345') → '34512'	'34512'	OK	
left2('Chocolate') → 'ocolateCh'	'ocolateCh'	OK	
left2('bricks') → 'icksbr'	'icksbr'	OK	
other tests		OK	

All Correct

Good job -- problem solved. You can see our solution as an alternative.

See Our Solution

next | chance

Python > String-1

done page

String-2 > double_char

prev | next | chance

Given a string, return a string where for every char in the original, there are two chars.

double_char('The') → 'TThhee'
double_char('AAbb') → 'AAAAbbbb'
double_char('Hi-There') → 'HHii--TThheerree'

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def double_char(s):
    result = ""
    for c in s:
        result += c * 2
    return result
```

Expected	Run		
double_char('The') → 'TThhee'	'TThhee'	OK	
double_char('AAbb') → 'AAAAbbbb'	'AAAAbbbb'	OK	
double_char('Hi-There') → 'HHii--TThheerree'	'HHii--TThheerree'	OK	
double_char('Word!') → 'WWoorrdd!!'	'WWoorrdd!!'	OK	
double_char('!!') → '!!!!'	'!!!!'	OK	
double_char('') → ''	''	OK	
double_char('a') → 'aa'	'aa'	OK	
double_char('.') → '..'	'..'	OK	
double_char('aa') → 'aaaa'	'aaaa'	OK	
other tests		OK	

All Correct

String-2 > count_hi

prev | next | chance

Return the number of times that the string "hi" appears anywhere in the given string.

count_hi('abc hi ho') → 1
count_hi('ABChi hi') → 2
count_hi('hihi') → 2

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def count_hi(s):
    return s.count("hi")
```

Expected	Run	
count_hi('abc hi ho') → 1	1	OK
count_hi('ABChi hi') → 2	2	OK
count_hi('hihi') → 2	2	OK
count_hi('hiHIhi') → 2	2	OK
count_hi('') → 0	0	OK
count_hi('h') → 0	0	OK
count_hi('hi') → 1	1	OK
count_hi('Hi is no HI on ahI') → 0	0	OK
count_hi('hiho not HOHIhi') → 2	2	OK
other tests		OK

All Correct

Good job -- problem solved. You can see our solution as an alternative.

Java | Python

String-2 > count_code

prev | next | chance

Return the number of times that the string "code" appears anywhere in the given string, except we'll accept any letter for the 'd', so "cope" and "cooe" count.

count_code('aaacodebbb') → 1
count_code('codexxcode') → 2
count_code('cozexxcope') → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def count_code(s):
    count = 0
    for i in range(len(s) - 3):
        if s[i:i+2] == "co" and s[i+3] == "e":
            count += 1
    return count
```

Go

Expected	Run	
count_code('aaacodebbb') → 1	1	OK
count_code('codexxcode') → 2	2	OK
count_code('cozexxcope') → 2	2	OK
count_code('cozfxxcope') → 1	1	OK
count_code('xxcozeyycop') → 1	1	OK
count_code('cozcop') → 0	0	OK
count_code('abcxyz') → 0	0	OK
count_code('code') → 1	1	OK
count_code('ode') → 0	0	OK
count_code('c') → 0	0	OK
count_code('') → 0	0	OK
count_code('AAcodeBBcoleCCcccoreDD') → 3	3	OK
count_code('AAcodeBBcoleCCcccorfDD') → 2	2	OK
count_code('coAcodeBcoleccoreDD') → 3	3	OK
other tests		OK

All Correct

String-2 > end_other

prev | next | chance

Given two strings, return True if either of the strings appears at the very end of the other string, ignoring upper/lower case differences (in other words, the computation should not be "case sensitive"). Note: s.lower() returns the lowercase version of a string.

end_other('Hiabc', 'abc') → True
end_other('AbC', 'HiaBc') → True
end_other('abc', 'abXabc') → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def end_other(a, b):
    a = a.lower()
    b = b.lower()
    return a.endswith(b) or b.endswith(a)
```

Expected	Run		
end_other('Hiabc', 'abc') → True	True	OK	
end_other('AbC', 'HiaBc') → True	True	OK	
end_other('abc', 'abXabc') → True	True	OK	
end_other('Hiabc', 'abcd') → False	False	OK	
end_other('Hiabc', 'bc') → True	True	OK	
end_other('Hiabcx', 'bc') → False	False	OK	
end_other('abc', 'abc') → True	True	OK	
end_other('xyz', '12xyz') → True	True	OK	
end_other('yz', '12xz') → False	False	OK	
end_other('Z', '12xz') → True	True	OK	
end_other('12', '12') → True	True	OK	
end_other('abcXYZ', 'abcDEF') → False	False	OK	
end_other('ab', 'ab12') → False	False	OK	
end_other('ab', '12ab') → True	True	OK	
other tests		OK	

String-2 > xyz_there

prev | next | chance

Return True if the given string contains an appearance of "xyz" where the xyz is not directly preceeded by a period (.). So "xxyz" counts but "x.xyz" does not.

xyz_there('abcxyz') → True
xyz_there('abc.xyz') → False
xyz_there('xyz.abc') → True

Go ...Save, Compile, Run (ctrl-enter)

```
def xyz_there(s):
    for i in range(len(s) - 2):
        if s[i:i+3] == "xyz":
            if i == 0 or s[i-1] != ".":
                return True
    return False
```

Expected	Run		
xyz_there('abcxyz') → True	True	OK	
xyz_there('abc.xyz') → False	False	OK	
xyz_there('xyz.abc') → True	True	OK	
xyz_there('abcxy') → False	False	OK	
xyz_there('xyz') → True	True	OK	
xyz_there('xy') → False	False	OK	
xyz_there('x') → False	False	OK	
xyz_there('') → False	False	OK	
xyz_there('abc.xyzxyz') → True	True	OK	
xyz_there('abc.xxyz') → True	True	OK	
xyz_there('.xyz') → False	False	OK	
xyz_there('12.xyz') → False	False	OK	
xyz_there('12xyz') → True	True	OK	
xyz_there('1.xyz.xyz2.xyz') → False	False	OK	
other tests		OK	

String-2 > cat_dog

prev | next | chance

Return True if the string "cat" and "dog" appear the same number of times in the given string.

cat_dog('catdog') → True
cat_dog('catcat') → False
cat_dog('1cat1cadodog') → True

Go ...Save, Compile, Run (ctrl-enter)

```
def cat_dog(s):
    cat = 0
    dog = 0

    for i in range(len(s) - 2):
        if s[i:i+3] == "cat":
            cat += 1
        if s[i:i+3] == "dog":
            dog += 1

    return cat == dog
```

Expected	Run		
cat_dog('catdog') → True	True	OK	
cat_dog('catcat') → False	False	OK	
cat_dog('1cat1cadodog') → True	True	OK	
cat_dog('catxxdogxxxdog') → False	False	OK	
cat_dog('catxdogxdogxcat') → True	True	OK	
cat_dog('catxdogxdogxca') → False	False	OK	
cat_dog('dogdogcat') → False	False	OK	
cat_dog('dogogcat') → True	True	OK	
cat_dog('dog') → False	False	OK	
cat_dog('cat') → False	False	OK	
cat_dog('ca') → True	True	OK	
cat_dog('c') → True	True	OK	
cat_dog('') → True	True	OK	
other tests		OK	

All Correct

List-1 > first_last6

prev | next | chance

Given an array of ints, return True if 6 appears as either the first or last element in the array. The array will be length 1 or more.

first_last6([1, 2, 6]) → True
first_last6([6, 1, 2, 3]) → True
first_last6([13, 6, 1, 2, 3]) → False

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def first_last6(nums):
  if nums[0] == 6 or nums[-1]==6:
    return True
  return False
```

Expected	Run		
first_last6([1, 2, 6]) → True	True	OK	
first_last6([6, 1, 2, 3]) → True	True	OK	
first_last6([13, 6, 1, 2, 3]) → False	False	OK	
first_last6([13, 6, 1, 2, 6]) → True	True	OK	
first_last6([3, 2, 1]) → False	False	OK	
first_last6([3, 6, 1]) → False	False	OK	
first_last6([3, 6]) → True	True	OK	
first_last6([6]) → True	True	OK	
first_last6([3]) → False	False	OK	
first_last6([5, 6]) → True	True	OK	
first_last6([5, 5]) → False	False	OK	
first_last6([1, 2, 3, 4, 6]) → True	True	OK	
first_last6([1, 2, 3, 4]) → False	False	OK	
other tests		OK	

Java | Python

List-1 > same_first_last

prev | next | chance

Given an array of ints, return True if the array is length 1 or more, and the first element and the last element are equal.

same_first_last([1, 2, 3]) → False
same_first_last([1, 2, 3, 1]) → True
same_first_last([1, 2, 1]) → True

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def same_first_last(nums):
    return len(nums) >= 1 and nums[0] == nums[-1]
```

Go

Expected	Run	
same_first_last([1, 2, 3]) → False	False	OK
same_first_last([1, 2, 3, 1]) → True	True	OK
same_first_last([1, 2, 1]) → True	True	OK
same_first_last([7]) → True	True	OK
same_first_last([]) → False	False	OK
same_first_last([1, 2, 3, 4, 5, 1]) → True	True	OK
same_first_last([1, 2, 3, 4, 5, 13]) → False	False	OK
same_first_last([13, 2, 3, 4, 5, 13]) → True	True	OK
same_first_last([7, 7]) → True	True	OK
other tests		OK

All Correct

Good job -- problem solved. You can see our solution as an alternative.

See Our Solution

next | chance

Python > List-1

List-1 > make_pi

prev | next | chance

Return an int array length 3 containing the first 3 digits of pi, {3, 1, 4}.

make_pi() → [3, 1, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_pi():
    return [3, 1, 4]
```

Expected	Run	
make_pi() → [3, 1, 4]	[3, 1, 4]	OK

All Correct

next | chance

Python > List-1

done page

Code is saved so long as this session is active. Create an account al to save code past this session.

Your progress graph for this problem

Java | Python

List-1 > common_end

prev | next | chance

Given 2 arrays of ints, a and b, return True if they have the same first element or they have the same last element. Both arrays will be length 1 or more.

common_end([1, 2, 3], [7, 3]) → True
common_end([1, 2, 3], [7, 3, 2]) → False
common_end([1, 2, 3], [1, 3]) → True

Go ...Save, Compile, Run (ctrl-enter)

```
def common_end(a, b):
  return a[0]==b[0] or a[-1] == b[-1]
```

Expected	Run	
common_end([1, 2, 3], [7, 3]) → True	True	OK
common_end([1, 2, 3], [7, 3, 2]) → False	False	OK
common_end([1, 2, 3], [1, 3]) → True	True	OK
common_end([1, 2, 3], [1]) → True	True	OK
common_end([1, 2, 3], [2]) → False	False	OK
other tests		OK

All Correct

next | chance

Python > List-1

List-1 > sum3

prev | next | chance

Given an array of ints length 3, return the sum of all the elements.

sum3([1, 2, 3]) → 6
sum3([5, 11, 2]) → 18
sum3([7, 0, 0]) → 7

Go ...Save, Compile, Run (ctrl-enter)

```
def sum3(nums):
  return sum(nums)
```

Expected	Run		
sum3([1, 2, 3]) → 6	6	OK	
sum3([5, 11, 2]) → 18	18	OK	
sum3([7, 0, 0]) → 7	7	OK	
sum3([1, 2, 1]) → 4	4	OK	
sum3([1, 1, 1]) → 3	3	OK	
sum3([2, 7, 2]) → 11	11	OK	

All Correct

next | chance

List-1 > rotate_left3

prev | next | chance

Given an array of ints length 3, return an array with the elements "rotated left" so {1, 2, 3} yields {2, 3, 1}.

rotate_left3([1, 2, 3]) → [2, 3, 1]
rotate_left3([5, 11, 9]) → [11, 9, 5]
rotate_left3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def rotate_left3(nums):
    return [nums[1], nums[2], nums[0]]
```

Expected	Run		
rotate_left3([1, 2, 3]) → [2, 3, 1]	[2, 3, 1]	OK	
rotate_left3([5, 11, 9]) → [11, 9, 5]	[11, 9, 5]	OK	
rotate_left3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK	
rotate_left3([1, 2, 1]) → [2, 1, 1]	[2, 1, 1]	OK	
rotate_left3([0, 0, 1]) → [0, 1, 0]	[0, 1, 0]	OK	
other tests		OK	

All Correct

next | chance

Python > List-1

List-1 > reverse3

prev | next | chance

Given an array of ints length 3, return a new array with the elements in reverse order, so {1, 2, 3} becomes {3, 2, 1}.

reverse3([1, 2, 3]) → [3, 2, 1]
reverse3([5, 11, 9]) → [9, 11, 5]
reverse3([7, 0, 0]) → [0, 0, 7]

Go ...Save, Compile, Run (ctrl-enter)

```
def reverse3(nums):
  return nums[::-1]
```

Expected	Run		
reverse3([1, 2, 3]) → [3, 2, 1]	[3, 2, 1]	OK	
reverse3([5, 11, 9]) → [9, 11, 5]	[9, 11, 5]	OK	
reverse3([7, 0, 0]) → [0, 0, 7]	[0, 0, 7]	OK	
reverse3([2, 1, 2]) → [2, 1, 2]	[2, 1, 2]	OK	
reverse3([1, 2, 1]) → [1, 2, 1]	[1, 2, 1]	OK	
reverse3([2, 11, 3]) → [3, 11, 2]	[3, 11, 2]	OK	
reverse3([0, 6, 5]) → [5, 6, 0]	[5, 6, 0]	OK	
reverse3([7, 2, 3]) → [3, 2, 7]	[3, 2, 7]	OK	
other tests		OK	

All Correct

List-1 > max_end3

prev | next | chance

Given an array of ints length 3, figure out which is larger, the first or last element in the array, and set all the other elements to be that value. Return the changed array.

max_end3([1, 2, 3]) → [3, 3, 3]
max_end3([11, 5, 9]) → [11, 11, 11]
max_end3([2, 11, 3]) → [3, 3, 3]

Go ...Save, Compile, Run (ctrl-enter)

```
def max_end3(nums):
    m = max(nums[0], nums[2])
    return [m, m, m]
```

Expected	Run		
max_end3([1, 2, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 5, 9]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 11, 3]) → [3, 3, 3]	[3, 3, 3]	OK	
max_end3([11, 3, 3]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([3, 11, 11]) → [11, 11, 11]	[11, 11, 11]	OK	
max_end3([2, 2, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([2, 11, 2]) → [2, 2, 2]	[2, 2, 2]	OK	
max_end3([0, 0, 1]) → [1, 1, 1]	[1, 1, 1]	OK	
other tests		OK	

All Correct

List-1 > sum2

prev | next | chance

Given an array of ints, return the sum of the first 2 elements in the array. If the array length is less than 2, just sum up the elements that exist, returning 0 if the array is length 0.

sum2([1, 2, 3]) → 3
sum2([1, 1]) → 2
sum2([1, 1, 1, 1]) → 2

Go ...Save, Compile, Run (ctrl-enter)

```
def sum2(nums):
    if len(nums) == 0:
        return 0
    elif len(nums) == 1:
        return nums[0]
    else:
        return nums[0] + nums[1]
```

Expected	Run		
sum2([1, 2, 3]) → 3	3	OK	
sum2([1, 1]) → 2	2	OK	
sum2([1, 1, 1, 1]) → 2	2	OK	
sum2([1, 2]) → 3	3	OK	
sum2([1]) → 1	1	OK	
sum2([]) → 0	0	OK	
sum2([4, 5, 6]) → 9	9	OK	
sum2([4]) → 4	4	OK	
other tests		OK	

All Correc

next | chance

List-1 > middle_way

prev | next | chance

Given 2 int arrays, a and b, each length 3, return a new array length 2 containing their middle elements.

middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]

Go ...Save, Compile, Run (ctrl-enter)

```
def middle_way(a, b):
  return [a[1], b[1]]
```

Expected	Run		
middle_way([1, 2, 3], [4, 5, 6]) → [2, 5]	[2, 5]	OK	
middle_way([7, 7, 7], [3, 8, 0]) → [7, 8]	[7, 8]	OK	
middle_way([5, 2, 9], [1, 4, 5]) → [2, 4]	[2, 4]	OK	
middle_way([1, 9, 7], [4, 8, 8]) → [9, 8]	[9, 8]	OK	
middle_way([1, 2, 3], [3, 1, 4]) → [2, 1]	[2, 1]	OK	
middle_way([1, 2, 3], [4, 1, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

All Correct

List-1 > make_ends

prev | next | chance

Given an array of ints, return a new array length 2 containing the first and last elements from the original array. The original array will be length 1 or more.

make_ends([1, 2, 3]) → [1, 3]
make_ends([1, 2, 3, 4]) → [1, 4]
make_ends([7, 4, 6, 2]) → [7, 2]

Go ...Save, Compile, Run (ctrl-enter)

```
def make_ends(nums):
    return [nums[0], nums[-1]]
```

Expected	Run		
make_ends([1, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([1, 2, 3, 4]) → [1, 4]	[1, 4]	OK	
make_ends([7, 4, 6, 2]) → [7, 2]	[7, 2]	OK	
make_ends([1, 2, 2, 2, 2, 2, 2, 3]) → [1, 3]	[1, 3]	OK	
make_ends([7, 4]) → [7, 4]	[7, 4]	OK	
make_ends([7]) → [7, 7]	[7, 7]	OK	
make_ends([5, 2, 9]) → [5, 9]	[5, 9]	OK	
make_ends([2, 3, 4, 1]) → [2, 1]	[2, 1]	OK	
other tests		OK	

List-1 > has23

prev | next | chance

Given an int array length 2, return True if it contains a 2 or a 3.

has23([2, 5]) → True
has23([4, 3]) → True
has23([4, 5]) → False

Go ...Save, Compile, Run (ctrl-enter)

```
def has23(nums):
    return nums[0] == 2 or nums[0] == 3 or nums[1] == 2 or nums[1] == 3
```

Expected	Run		
has23([2, 5]) → True	True	OK	
has23([4, 3]) → True	True	OK	
has23([4, 5]) → False	False	OK	
has23([2, 2]) → True	True	OK	
has23([3, 2]) → True	True	OK	
has23([3, 3]) → True	True	OK	
has23([7, 7]) → False	False	OK	
has23([3, 9]) → True	True	OK	
has23([9, 5]) → False	False	OK	
other tests		OK	

Java | Python

List-2 > count_evens

prev | next | chance

Return the number of even ints in the given array. Note: the % "mod" operator computes the remainder, e.g. 5 % 2 is 1.

count_evens([2, 1, 2, 3, 4]) → 3
count_evens([2, 2, 0]) → 3
count_evens([1, 3, 5]) → 0

Go ...Save, Compile, Run (ctrl-enter) Show Hint

```
def count_evens(nums):
  count =0
  for i in range(len(nums)):
    if nums[i]%2==0:
      count+=1
  return count
```

Expected	Run		
count_evens([2, 1, 2, 3, 4]) → 3	3	OK	
count_evens([2, 2, 0]) → 3	3	OK	
count_evens([1, 3, 5]) → 0	0	OK	
count_evens([]) → 0	0	OK	
count_evens([11, 9, 0, 1]) → 1	1	OK	
count_evens([2, 11, 9, 0]) → 2	2	OK	
count_evens([2]) → 1	1	OK	
count_evens([2, 5, 12]) → 2	2	OK	
other tests		OK	

All Correct

Good job -- problem solved. You can see our solution as an alternative.

See Our Solution

next | chance

List-2 > big_diff

prev | next | chance

Given an array length 1 or more of ints, return the difference between the largest and smallest values in the array. Note: the built-in min(v1, v2) and max(v1, v2) functions return the smaller or larger of two values.

big_diff([10, 3, 5, 6]) → 7
big_diff([7, 2, 10, 9]) → 8
big_diff([2, 10, 7, 2]) → 8

Go ...Save, Compile, Run (ctrl-enter)

```python
def big_diff(nums):
    return max(nums) - min(nums)
```

Expected	Run	
big_diff([10, 3, 5, 6]) → 7	7	OK
big_diff([7, 2, 10, 9]) → 8	8	OK
big_diff([2, 10, 7, 2]) → 8	8	OK
big_diff([2, 10]) → 8	8	OK
big_diff([10, 2]) → 8	8	OK
big_diff([10, 0]) → 10	10	OK
big_diff([2, 3]) → 1	1	OK
big_diff([2, 2]) → 0	0	OK
big_diff([2]) → 0	0	OK
big_diff([5, 1, 6, 1, 9, 9]) → 8	8	OK
big_diff([7, 6, 8, 5]) → 3	3	OK
big_diff([7, 7, 6, 8, 5, 5, 6]) → 3	3	OK
other tests		OK

Java | Python

List-2 > centered_average

prev | next | chance

Return the "centered" average of an array of ints, which we'll say is the mean average of the values, except ignoring the largest and smallest values in the array. If there are multiple copies of the smallest value, ignore just one copy, and likewise for the largest value. Use int division to produce the final average. You may assume that the array is length 3 or more.

centered_average([1, 2, 3, 4, 100]) → 3
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5
centered_average([-10, -4, -2, -4, -2, 0]) → -3

Go ...Save, Compile, Run (ctrl-enter)

```python
def centered_average(nums):
    nums.remove(min(nums))
    nums.remove(max(nums))
    return sum(nums) // len(nums)
```

Expected	Run	
centered_average([1, 2, 3, 4, 100]) → 3	3	OK
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5	5	OK
centered_average([-10, -4, -2, -4, -2, 0]) → -3	-3	OK
centered_average([5, 3, 4, 6, 2]) → 4	4	OK
centered_average([5, 3, 4, 0, 100]) → 4	4	OK
centered_average([100, 0, 5, 3, 4]) → 4	4	OK
centered_average([4, 0, 100]) → 4	4	OK
centered_average([0, 2, 3, 4, 100]) → 3	3	OK
centered_average([1, 1, 100]) → 1	1	OK
centered_average([7, 7, 7]) → 7	7	OK
centered_average([1, 7, 8]) → 7	7	OK
centered_average([1, 1, 99, 99]) → 50	50	OK
centered_average([1000, 0, 1, 99]) → 50	50	OK
centered_average([4, 4, 4, 4, 5]) → 4	4	OK
centered_average([4, 4, 4, 1, 5]) → 4	4	OK
centered_average([6, 4, 8, 12, 3]) → 6	6	OK
other tests		OK

List-2 > sum13

prev | next | chance

Return the sum of the numbers in the array, returning 0 for an empty array. Except the number 13 is very unlucky, so it does not count and numbers that come immediately after a 13 also do not count.

sum13([1, 2, 2, 1]) → 6
sum13([1, 1]) → 2
sum13([1, 2, 2, 1, 13]) → 6

Go ...Save, Compile, Run (ctrl-enter)

```python
def sum13(nums):
    total = 0
    i = 0

    while i < len(nums):
        if nums[i] == 13:
            i += 2
        else:
            total += nums[i]
            i += 1

    return total
```

Expected	Run	
sum13([1, 2, 2, 1]) → 6	6	OK
sum13([1, 1]) → 2	2	OK
sum13([1, 2, 2, 1, 13]) → 6	6	OK
sum13([1, 2, 13, 2, 1, 13]) → 4	4	OK
sum13([13, 1, 2, 13, 2, 1, 13]) → 3	3	OK
sum13([]) → 0	0	OK
sum13([13]) → 0	0	OK
sum13([13, 13]) → 0	0	OK
sum13([13, 0, 13]) → 0	0	OK
sum13([13, 1, 13]) → 0	0	OK
sum13([5, 7, 2]) → 14	14	OK
sum13([5, 13, 2]) → 5	5	OK
sum13([0]) → 0	0	OK
sum13([13, 0]) → 0	0	OK
other tests		OK

List-2 > sum67

prev | next | chance

Return the sum of the numbers in the array, except ignore sections of numbers starting with a 6 and extending to the next 7 (every 6 will be followed by at least one 7). Return 0 for no numbers.

sum67([1, 2, 2]) → 5
sum67([1, 2, 2, 6, 99, 99, 7]) → 5
sum67([1, 1, 6, 7, 2]) → 4

Go ...Save, Compile, Run (ctrl-enter)

```
def sum67(nums):
    total = 0
    skip = False

    for n in nums:
        if n == 6:
            skip = True
        elif n == 7 and skip:
            skip = False
        elif not skip:
            total += n

    return total
```

Expected	Run		
sum67([1, 2, 2]) → 5	5	OK	
sum67([1, 2, 2, 6, 99, 99, 7]) → 5	5	OK	
sum67([1, 1, 6, 7, 2]) → 4	4	OK	
sum67([1, 6, 2, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK	
sum67([1, 6, 2, 6, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK	
sum67([2, 7, 6, 2, 6, 7, 2, 7]) → 18	18	OK	
sum67([2, 7, 6, 2, 6, 2, 7]) → 9	9	OK	
sum67([1, 6, 7, 7]) → 8	8	OK	
sum67([6, 7, 1, 6, 7, 7]) → 8	8	OK	
sum67([6, 8, 1, 6, 7]) → 0	0	OK	
sum67([]) → 0	0	OK	
sum67([6, 7, 11]) → 11	11	OK	
sum67([11, 6, 7, 11]) → 22	22	OK	
sum67([2, 2, 6, 7, 7]) → 11	11	OK	
other tests		OK	

All Correct

List-2 > has22

prev | next | chance

Given an array of ints, return True if the array contains a 2 next to a 2 somewhere.

has22([1, 2, 2]) → True
has22([1, 2, 1, 2]) → False
has22([2, 1, 2]) → False

Go ...Save, Compile, Run (ctrl-enter)

```
def has22(nums):
    for i in range(len(nums) - 1):
        if nums[i] == 2 and nums[i+1] == 2:
            return True
    return False
```

Expected	Run		
has22([1, 2, 2]) → True	True	OK	
has22([1, 2, 1, 2]) → False	False	OK	
has22([2, 1, 2]) → False	False	OK	
has22([2, 2, 1, 2]) → True	True	OK	
has22([1, 3, 2]) → False	False	OK	
has22([1, 3, 2, 2]) → True	True	OK	
has22([2, 3, 2, 2]) → True	True	OK	
has22([4, 2, 4, 2, 2, 5]) → True	True	OK	
has22([1, 2]) → False	False	OK	
has22([2, 2]) → True	True	OK	
has22([2]) → False	False	OK	
has22([]) → False	False	OK	
has22([3, 3, 2, 2]) → True	True	OK	
has22([5, 2, 5, 2]) → False	False	OK	
other tests		OK	